

Laboratório 3: Dados relacionais e operações do tipo join

AUTHOR

Noemi de Souza Leone

HORÁRIO DE COPILAÇÃO

```
Sys.setenv(TZ = "America/Sao_Paulo")
data_hora <- format(Sys.time(), "%d/%m/%Y às %H:%M:%S")
data_hora
```

```
[1] "29/08/2026 às 23:27:59"
```

Carregmento de Pacotes

INSTRUÇÃO 01

Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

```
# Verificando conteúdo do arquivo airlines.csv
airlines <- read_csv("airlines.csv")
```

```
Rows: 14 Columns: 2
```

```
— Column specification —————
```

```
Delimiter: ","
```

```
chr (2): IATA_CODE, AIRLINE
```

• Use `spec()` to retrieve the full column specification for this data.

• Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
airlines
```

```
# A tibble: 14 × 2
```

```
  IATA_CODE AIRLINE
```

```
  <chr>      <chr>
```

```
1 UA      United Air Lines Inc.
2 AA      American Airlines Inc.
3 US      US Airways Inc.
4 F9      Frontier Airlines Inc.
5 B6      JetBlue Airways
6 OO      Skywest Airlines Inc.
7 AS      Alaska Airlines Inc.
8 NK      Spirit Air Lines
9 WN      Southwest Airlines Co.
```

```

10 DL      Delta Air Lines Inc.
11 EV      Atlantic Southeast Airlines
12 HA      Hawaiian Airlines Inc.
13 MQ      American Eagle Airlines Inc.
14 VX      Virgin America

```

```

# Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE
airports <- read_csv(
  "airports.csv",
  col_select = c(
    IATA_CODE,
    CITY,
    STATE,
    LATITUDE,
    LONGITUDE
  )
)

```

Rows: 322 Columns: 5

— Column specification —

Delimiter: ","

chr (3): IATA_CODE, CITY, STATE

dbl (2): LATITUDE, LONGITUDE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

airports

A tibble: 322 × 5

	IATA_CODE	CITY	STATE	LATITUDE	LONGITUDE
	<chr>	<chr>	<chr>	<dbl>	<dbl>
1	ABE	Allentown	PA	40.7	-75.4
2	ABI	Abilene	TX	32.4	-99.7
3	ABQ	Albuquerque	NM	35.0	-107.
4	ABR	Aberdeen	SD	45.4	-98.4
5	ABY	Albany	GA	31.5	-84.2
6	ACK	Nantucket	MA	41.3	-70.1
7	ACT	Waco	TX	31.6	-97.2
8	ACV	Arcata/Eureka	CA	41.0	-124.
9	ACY	Atlantic City	NJ	39.5	-74.6
10	ADK	Adak	AK	51.9	-177.

i 312 more rows

#Para arquivos de voos: ##Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino; ##Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```

getStats <- function(input, pos) {
  input |>
    filter(
      !is.na(DESTINATION_AIRPORT),

```

```

    !is.na(ARRIVAL_DELAY)
  ) |>
  group_by(DESTINATION_AIRPORT) |>
  summarise(
    soma = sum(ARRIVAL_DELAY),
    n = n(),
    .groups = "drop"
  )
}

callback <- DataFrameCallback$new(getStats)

```

```

flights <- read_csv_chunked(
  "flights.csv",
  callback = callback,
  chunk_size = 1000000,
  col_types = cols_only(
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)
flights

```

```

# A tibble: 2,181 × 3
  DESTINATION_AIRPORT  soma    n
  <chr>              <dbl> <int>
1 ABE                 3413   350
2 ABI                 5926   446
3 ABQ                20448  3241
4 ABR                 -645   124
5 ABY                 1339   169
6 ACT                 4436   271
7 ACV                 2748   294
8 ACY                 9069   701
9 ADK                 -293    19
10 ADQ                 72     59
# i 2,171 more rows

```

Instrução 02

```

# Aeroportos presentes em flights, mas ausentes em airports
flights |>
  anti_join(
    airports,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  )|>
  select(DESTINATION_AIRPORT)

```

```

# A tibble: 307 × 1
  DESTINATION_AIRPORT
  <chr>

```

```
1 10135
2 10136
3 10140
4 10141
5 10146
6 10154
7 10155
8 10157
9 10158
10 10165
# i 297 more rows
```

```
# Incluindo CITY e STATE em flights
flights <- flights |>
  left_join(
    airports |>
      select(IATA_CODE, CITY, STATE),
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  )
```

INSTRUÇÃO 03

Quantidade de Aeroportos por estado

```
flights |>
  count(STATE, name = "N_AEROPORTOS", sort = TRUE)
```

```
# A tibble: 55 × 2
  STATE N_AEROPORTOS
  <chr>      <int>
1 <NA>         307
2 TX           144
3 CA           121
4 AK           102
5 FL           102
6 MI            90
7 NY            80
8 CO            57
9 NC            48
10 ND           48
# i 45 more rows
```

INSTRUÇÃO 04

Carregando o pacote

```
flights <- flights |>
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) |>
```

```
mutate(atraso_medio = soma / n)

# Criando o mapa
this <- leaflet(flights) |>
  addTiles() |>
  addCircleMarkers(
    radius = ~atraso_medio/5,
    color = ~ifelse(atraso_medio > 30, "red", "blue"),
    popup = ~paste(DESTINATION_AIRPORT, ":", round(atraso_medio, 1), "min")
  )
```

Assuming "LONGITUDE" and "LATITUDE" are longitude and latitude, respectively

Warning in validateCoords(lng, lat, funcName): Data contains 325 rows with either missing or invalid lat/lon values and will be ignored

this

